

Technical Architecture & System Documentation

A precise reference for the Orbyte platform: backend services, the retrieval-augmented generation pipeline, data connectors, frontend applications, and deployment infrastructure — reflecting the system as currently built and wired, not upstream defaults.

Scope

Backend · AI/RAG · Frontend · Infra

Basis

Onyx (OSS) core, reduced & rebranded

Date

July 19, 2026

§ Contents

01	Overview	What Orbyte is and its core components
02	System Architecture	Services and data flow
03	Backend	Stack, modules, auth, database, workers, search index
04	AI & Retrieval Pipeline	Chunking, embeddings, hybrid search, LLMs, agent tools
05	Data Connectors	Active sources and sync mechanics
06	Frontend	Stack, applications, admin dashboard
07	Security & Access Control	AuthN/AuthZ, permissions, encryption
08	Infrastructure & Deployment	Container services and topology
09	Technology Stack Reference	Consolidated version table

01 Overview

Product & component summary

Orbyte is an AI platform that lets users chat with an LLM grounded in their own connected data. It is built on a reduced, rebranded fork of the open-source Onyx platform: the full connector and admin surface of upstream Onyx has been trimmed to a focused core — chat, retrieval-augmented generation (RAG), a small set of first-party connectors, and access-controlled administration — while the underlying engine (indexing pipeline, hybrid search, multi-provider LLM routing, permissioning) remains fully intact.

Product surfaces

Surface	Route	Purpose
Chat	<code>/app</code>	Primary conversational interface: agents, streamed answers, citations, per-agent tools, Deep Research mode.
Craft	<code>/craft</code>	Sandboxed agent/app-building workspace — an in-product coding-agent environment with a live sandbox, file/diff/search tool cards, and approvals.
NRF	<code>/nrf</code>	Companion browser-extension surface: new-tab-page replacement and side-panel chat.
Admin	<code>/admin</code>	Configuration of LLMs, connectors, agents, users and groups.
Enterprise	<code>/ee</code>	Subscription-gated features: billing, whitelabel theming, usage analytics, query history.

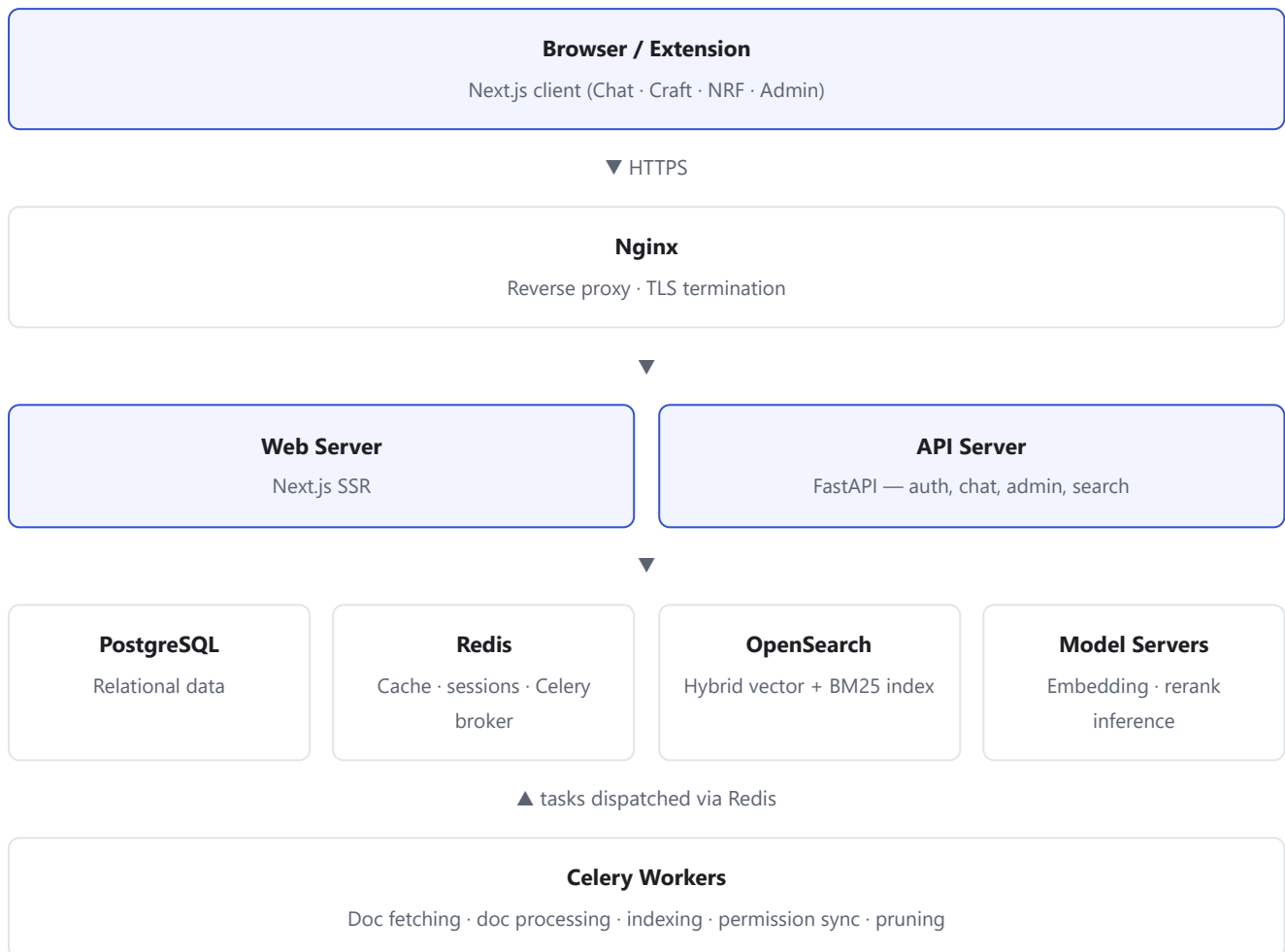
Core components at a glance

Component	Technology	Role
Web application	Next.js / React	All user-facing and admin UI
API server	FastAPI (Python)	Auth, chat, connector, admin, and search endpoints
Background workers	Celery + Redis	Connector sync, document processing, indexing, permission sync
Relational database	PostgreSQL	Users, permissions, connectors, chat history, metadata
Search index	OpenSearch	Hybrid vector + keyword (BM25) retrieval over document chunks
ML inference servers	SentenceTransformers / PyTorch	Embedding & reranking, isolated from the API server

Cache / broker	Redis	Sessions, query-embedding cache, Celery message broker
Object storage	MinIO (S3-compatible)	Uploaded files and generated artifacts
Sandbox	Docker-based code interpreter	Isolated code execution for the Python tool and Craft

02 System Architecture

Services & data flow



Indexing path

Celery Beat schedule → connector fetch → filter / dedup → chunk
→ embed (model server) → write to OpenSearch + PostgreSQL

Query path

Client request → API server (auth + permission check) → embed query
→ hybrid search in OpenSearch (permission-filtered) → optional rerank
→ prompt assembly → LLM (streamed) → citation mapping → SSE to client

Note. Permission filtering is applied *inside* the OpenSearch query itself (an `external_access` field on every document), not as a post-processing step — unauthorized chunks are never returned by the index.

3.1 — Core stack

Layer	Technology	Version
Language / runtime	Python	3.13
Web framework	FastAPI	0.133
ORM	SQLAlchemy	2.0.50
Migrations	Alembic	1.18
Validation	Pydantic	2.11
Task queue	Celery	5.5
LLM routing	LiteLLM	1.89
Search client	opensearch-py	3.2
ASGI server	Uvicorn	0.49
Embedding runtime	SentenceTransformers (PyTorch)	—

3.2 — Application modules

The backend is organized as a single Python package (`onyx/`) of focused modules. All listed modules below are actively wired into the running application (imported and registered from `main.py`), unless marked otherwise.

Module	Responsibility
<code>auth/</code>	Basic, Google OAuth, OIDC and SAML 2.0 authentication
<code>chat/</code>	Chat session/message handling, streaming response orchestration
<code>connectors/</code>	Source-system integrations (registry + sync interfaces)
<code>indexing/</code>	Chunking, contextual summarization, embedding orchestration
<code>document_index/</code>	Vector/keyword index abstraction (OpenSearch, Vespa)
<code>llm/</code>	Multi-provider LLM abstraction over LiteLLM
<code>tools/</code>	Agent tool registry and construction

<code>deep_research/</code>	Multi-step agentic research loop
<code>coding_agent/</code> · <code>sandbox_proxy/</code>	Craft coding-agent tool and its sandbox execution backend
<code>skills/</code>	Reusable AI skill definitions
<code>image_gen/</code>	Image generation tool + admin config
<code>voice/</code>	Speech-to-text / text-to-speech chat mode
<code>mcp_server/</code>	In-process Model Context Protocol tool client
<code>kg/</code>	Knowledge-graph admin configuration TOOL DISABLED
<code>background/</code>	Celery app, task definitions, worker entry points
<code>redis/</code>	Cache and distributed-lock abstractions
<code>file_store/</code>	Blob storage abstraction (MinIO / S3-compatible)
<code>db/</code>	SQLAlchemy models and queries (~154 model classes)
<code>server/</code>	All FastAPI route handlers
<code>prompts/</code>	System / user prompt templates
<code>tracing/</code>	OpenTelemetry request tracing
<code>ee/</code>	Enterprise-tier gated features (analytics, whitelabel, SCIM)

3.3 — Authentication & sessions

Four authentication backends are implemented and registered at startup: **Basic** (email + password, the default), **Google OAuth**, **OIDC**, and **SAML 2.0**. Sessions are stored server-side in Redis (not stateless JWT), default expiry 7 days, transported via an HTTP-only cookie — allowing instant session revocation. Role-based access control defines: Admin, Global Curator, Curator, Basic User, and Anonymous.

3.4 — Database

PostgreSQL is the system of record for everything except chunk vectors and full-text content. Schema evolution is managed through Alembic; the model layer (`db/models.py`) defines roughly 154 SQLAlchemy model classes. Central tables include users, connectors, credentials (encrypted), connector-credential pairs, documents, index attempts, chat sessions/messages, personas (AI agents), LLM provider configs, search settings, and document sets.

3.5 — Background processing

Celery, backed by Redis, runs dedicated worker pools per workload so each can be scaled independently:

Worker	Representative queues
Primary	<code>celery</code> — heartbeats, system checks
Doc-fetching	<code>connector_doc_fetching</code>
Doc-processing	<code>docprocessing</code> — chunk + embed + index
Heavy	<code>connector_pruning</code> , <code>connector_doc_permissions_sync</code> , <code>connector_external_group_sync</code>
Light	<code>vespa_metadata_sync</code> , <code>connector_deletion</code> , <code>doc_permissions_upsert</code>
User files	<code>user_file_processing</code> , <code>user_file_project_sync</code> , <code>user_file_delete</code>
Monitoring / Beat	<code>monitoring</code> , <code>scheduled_tasks</code> — periodic dispatch (cron-like)

3.6 — Search index

The system indexes into **OpenSearch 3.6** by default (`ENABLE_OPENSEARCH_INDEXING_FOR_ONYX=true` , `ONYX_DISABLE_VESPA=true`). The original Vespa index integration remains present in code but is disabled and not deployed — only an OpenSearch container is defined in the deployment stack. OpenSearch executes hybrid retrieval (dense vector similarity + BM25 keyword scoring) natively within a single query, with permission filters applied at the index layer.

04 AI & Retrieval Pipeline

Chunking · embeddings · hybrid search · agents

4.1 — Document ingestion pipeline

1. Filter
 - skip empty docs / docs exceeding max size
2. Vision pass
 - LLM-generated text description for image sections
3. De-duplicate
 - gate 1: updated_at unchanged → skip
 - gate 2: content-hash unchanged → skip
4. Upsert
 - document metadata, tags, permissions → PostgreSQL
5. Chunk
 - split into overlapping, sentence-aware chunks
6. Context (opt)
 - LLM prepends a short context summary per chunk
7. Embed
 - chunks → dense vectors via the embedding model server
8. Write
 - chunks + vectors + text → OpenSearch

4.2 — Chunking

Parameter	Value	Notes
Default max chunk size	512 tokens	Primary retrieval unit
Chunk overlap	0	Chunks are split on sentence/section boundaries rather than relying on overlap
Minimum content threshold	256 tokens	Below this, adjacent content is merged into a chunk
Mini-chunk size	150 tokens	Secondary, finer-grained chunks used for multi-vector precision retrieval
Large-chunk ratio	4×	Optional coarser chunk grouping when multipass + large-chunk mode is enabled
Contextual RAG	OFF BY DEFAULT	When enabled, gpt-4o-mini generates a context blurb prepended to each chunk before embedding, improving retrieval for context-dependent passages

4.3 — Embedding model

Setting	Value
Default model	nomic-ai/nomic-embed-text-v1
Vector dimensions	768 (environment-overridable)

Normalization	Enabled
Serving	Dedicated model server processes (indexing & inference are separate containers), isolated from the API server for independent scaling
Legacy model	<code>thenlper/gte-small</code> (384-dim) — retained for migration compatibility

Reranking uses a cross-encoder model that is configured per deployment through Search Settings in the admin panel — there is no single hardcoded default; an administrator selects and activates a reranker model explicitly.

4.4 — Hybrid retrieval

Every query combines **dense vector similarity** (semantic match) with **BM25** keyword scoring (exact term / proper-noun match) in a single OpenSearch query, so the system catches both conceptual questions and literal identifiers (error codes, product names) that pure vector search tends to miss. Results are permission-filtered inside the same query, then optionally re-scored by the cross-encoder reranker before the top candidates are passed to the LLM.

4.5 — LLM providers

All LLM calls route through a single provider abstraction built on LiteLLM. Twelve providers have first-class configuration support in the admin UI:

`Anthropic` · `OpenAI` · `Azure OpenAI` · `AWS Bedrock` · `VertexAI` · `Ollama` · `LM Studio` · `OpenRouter` · `LiteLLM Proxy` · `Bifrost` · `Nebius` `TokenFactory` · `OpenAI-compatible endpoint`

4.6 — Agent tools

Tool	Status	Description
Search	ACTIVE	Core RAG retrieval over indexed documents
Web Search	REQUIRES PROVIDER KEY	Serper, Google PSE, Brave, SearXNG, Firecrawl or Exa
Open URL	ACTIVE	Fetch and read a specific web page
Python / Code Interpreter	ACTIVE	Sandboxed code execution (dedicated container)
Coding Agent (Craft)	FLAGGED OFF	Full agentic build/sandbox workspace; disabled by default (<code>ENABLE_CRAFT=false</code>)
File Reader	ACTIVE	Reads uploaded/attached files into context
Memory	OPT-IN	Per-user persistent memory, enabled per user account

Custom / OpenAPI Actions	BACKEND ONLY	Functional in the engine; its admin UI entry point is currently removed from navigation
MCP Tools	ACTIVE	In-process Model Context Protocol client; the standalone MCP server container is disabled by default
Image Generation	REQUIRES CONFIG	Active only once an image-generation model is configured
Deep Research	ACTIVE	Not a tool call — a distinct multi-step agentic chat mode (plan → search → synthesize, iterated)
Knowledge Graph	DISABLED	Registered but its construction path is disabled; not reachable from any persona

4.7 — Chat request lifecycle

1. POST /chat/send-message (SSE connection opened)
2. Session validated against Redis-backed cookie
3. Persona + document-set scope resolved; user group memberships loaded
4. Query embedded (query-embedding cache in Redis, 15 min TTL)
5. Hybrid search executed in OpenSearch, permission- and scope-filtered
6. Optional cross-encoder reranking of top candidates
7. Prompt assembled: persona system prompt + history + retrieved chunks
8. LLM invoked via LiteLLM; tokens streamed back over SSE
9. Citation markers parsed and mapped to source documents
10. Assistant message + citations persisted to PostgreSQL

05 Data Connectors

Active sources & sync

The connector engine supports 54 source integrations in the backend registry. The current product intentionally exposes only five in the Add-Connector UI, keeping the surface area small and predictable:

Connector	Syncs	Auth
File Upload	Manually uploaded documents (PDF, Word, etc.)	—
Web Crawler	Public web pages from a seed URL	—
Google Drive	Docs, Sheets, Slides, and files in a Drive	OAuth 2.0
Confluence	Spaces and pages	API token / OAuth
Slack	Channels and message history	OAuth / bot token

Note. The remaining 49 connectors (Jira, Notion, GitHub, Zendesk, Salesforce, Gmail, S3, and others) remain fully implemented server-side and require no backend changes to re-enable — only removing them from the UI allowlist that currently restricts the Add-Connector page.

Sync mechanics

```
Celery Beat → connector due for sync? → doc-fetching queue
→ connector instantiated, documents pulled (full or incremental)
→ batches dispatched to doc-processing queue → chunk / embed / index
→ sync status recorded in PostgreSQL (IndexAttempt)
```

Most connectors support time-windowed incremental polling (only documents changed since the last successful sync); complex sources use a checkpointed, resumable sync. A separate permission-sync path mirrors source-system access-control lists into Orbyte so retrieval stays permission-aware.

6.1 — Core stack

Layer	Technology	Version
Framework	Next.js (App Router)	16.2
UI library	React	19.2
Language	TypeScript	5.9
Styling	Tailwind CSS	4.3
State management	Zustand	5.0
Data fetching	SWR	2.1
Forms & validation	Formik + Yup	—
Component primitives	Radix UI	—
Design system	Internal package <code>@onyx-ai/opal</code>	—
Package manager / runtime	Bun	—
Lint / format	oxlint / oxfmt	—
Testing	Jest, Testing Library, Playwright, Storybook	—

6.2 — Application surfaces

Chat (`/app`)

Primary conversational UI. Streams answers token-by-token over Server-Sent Events, renders inline citations, and exposes per-agent tools dynamically depending on which tools that agent is configured with. A Deep Research toggle is always available as a top-level chat mode.

Craft (`/craft`)

A sandboxed agent-building workspace — bash, diff, search, and web-fetch tool cards, a live sandbox preview, tool-call approvals, and sub-agents. Effectively an in-product coding-agent environment

NRF (`/nrf`)

The companion browser extension's surface: a new-tab-page replacement and a side-panel chat, offering a compact agent selector, model picker, and quick access into the full chat app.

Admin (`/admin`)

Configuration dashboard for LLMs, connectors, agents, and users — see 6.3.

for building custom apps. Gated off by default at the backend (`ENABLE_CRAFT=false`).

6.3 — Admin dashboard navigation

The navigation currently exposes:

- **Configuration** — LLM Models, Web Search, Chat Preferences
- **Agents & Actions** — Agents
- **Documents & Knowledge** — Indexing Status, Add Connector, Document Sets, Index Settings
- **Permissions** — Users, Groups

The following admin pages exist in code but have been deliberately removed from the navigation menu (reachable only by direct URL, if authorized): **Service Accounts**, **Query History**, **OpenAPI Actions**, **SSO Providers**, **SCIM**, and **Hooks/Webhooks**. This keeps the admin surface minimal without discarding the underlying functionality.

6.4 — Enterprise-tier UI

Billing, whitelabel theming, custom analytics, and query history live under `/ee` , gated by subscription tier. Below the required tier, matching entries render as disabled with an upgrade prompt rather than being hidden outright.

07 Security & Access Control

AuthN/AuthZ, permissions, encryption

7.1 — Authentication

- Passwords hashed via `fastapi-users` (bcrypt); configurable complexity policy (min/max length, character requirements)
- Sessions held server-side in Redis, default 7-day expiry, HTTP-only cookies (not readable by client-side JavaScript)
- Google OAuth, OIDC, and SAML 2.0 available alongside Basic auth for enterprise identity federation

7.2 — Authorization

Role-based access control governs administrative capability (Admin, Global Curator, Curator, Basic User, Anonymous). Independently, every indexed document carries an access-control field synchronized from its source system; retrieval queries are filtered by this field at the index layer, so a document a user cannot access is never returned — regardless of semantic relevance.

7.3 — Credential & data protection

- Connector credentials (OAuth tokens, API keys) encrypted at rest using a dedicated encryption key, independent of database access
- Credentials displayed in the admin UI are masked by default
- All secrets are supplied via environment variables, never hardcoded

08 Infrastructure & Deployment

The system is deployed as a set of Docker Compose services:

Service	Role	Status
api_server	FastAPI backend	ACTIVE
web_server	Next.js frontend	ACTIVE
background	Celery workers (all queues)	ACTIVE
relational_db	PostgreSQL 15.2	ACTIVE
opensearch	Search index, v3.6	ACTIVE
cache	Redis	ACTIVE
inference_model_server	Query-time embedding / rerank inference	ACTIVE
indexing_model_server	Indexing-time embedding inference	ACTIVE
minio	Object storage	ACTIVE
code-interpreter	Sandbox for code execution tools	ACTIVE
ollama	Local LLM runtime option	ACTIVE
nginx	Reverse proxy, TLS termination	ACTIVE
mcp_server	Standalone MCP server process	DISABLED

The ML inference workload is deliberately split from the API server into its own containers, so embedding/reranking throughput can be scaled (including onto GPU hardware) independently of API request handling. Kubernetes/Helm and Terraform deployment paths are retained alongside the Docker Compose stack for production environments.

09 Technology Stack Reference

Consolidated version table

Category	Technology	Version / Notes
Backend language	Python	3.13
Web framework	FastAPI	0.133
ORM / migrations	SQLAlchemy / Alembic	2.0.50 / 1.18
Validation	Pydantic	2.11
Task queue	Celery	5.5
LLM routing	LiteLLM	1.89
Relational database	PostgreSQL	15.2
Search index	OpenSearch	3.6
Cache / broker	Redis	5.0 client
Object storage	MinIO	S3-compatible
Embedding model	nomic-ai/nomic-embed-text-v1	768-dim
Embedding runtime	SentenceTransformers / PyTorch	—
Frontend framework	Next.js	16.2
UI library	React	19.2
Language	TypeScript	5.9
Styling	Tailwind CSS	4.3
Containerization	Docker / Docker Compose	—
Orchestration (production)	Kubernetes + Helm, Terraform	—
Reverse proxy	Nginx	—